Class 6

15 June 2026 10:34 PM

Case study : DESIGN OF BIRD CLASS

STEP 0 — Problem Setup (Baseline)

We are designing a system to model different types of birds.

Birds in the system:

- Sparrow
- Crow
- Pigeon
- Penguin
- Ostrich

Observing the Domain

Common behavior (present in all birds)

- eat()
- makeSound()

These behaviors are shared across all birds.

Non-common behavior

- Penguin, Ostrich → cannot fly
- Sparrow, Crow, Pigeon → can fly

Flying is **not a universal behavior**

Initial Design — Version 0

```
public class Bird {
    double weight;
    String color;
    double size;
    String type;

    void fly() {
        if (type.equals("Sparrow")) {
            System.out.println("Bird is flying");
        } else if (type.equals("Crow")) {
            System.out.println("Bird is flying");
        } else if (type.equals("Pigeon")) {
            System.out.println("Bird is flying");
        } else if (type.equals("Penguin")) {
            System.out.println("Bird cannot fly");
        } else if (type.equals("Ostrich")) {
            System.out.println("Bird cannot fly");
        }
    }

    void eat() {
        System.out.println("Bird is eating");
    }

    void makeSound() {
        System.out.println("Bird is making sound");
    }
```

```
}
```

Main Method

```
public class Main {
    public static void main(String[] args) {
        Bird sparrow = new Bird();
        sparrow.type = "Sparrow";
        sparrow.fly();

        Bird crow = new Bird();
        crow.type = "Crow";
        crow.fly();

        Bird pigeon = new Bird();
        pigeon.type = "Pigeon";
        pigeon.fly();

        Bird penguin = new Bird();
        penguin.type = "Penguin";
        penguin.fly();

        Bird ostrich = new Bird();
        ostrich.type = "Ostrich";
        ostrich.fly();
    }
}

/* *
    Output :
        Bird is flying
        Bird is flying
        Bird is flying
        Bird cannot fly
        Bird cannot fly
```

```
* */
```

Problems in This Design

- Poor readability (if-else logic)
- Hard to extend (requires modifying existing code)
- Difficult to test (logic tightly coupled)
- Low reusability (flying logic duplicated across birds)
- No clear separation of concerns
- Difficult for teams (multiple developers modifying same class)

Design Smells

1. Too Much Responsibilities in One Class

Bird is handling:

- Identity (type)
- Behavior decision (which bird does what)
- Behavior implementation (what the bird can do - fly logic, eat, sound)

2. Conditional Behavior (if-else based design)

if (type.equals(...))

- Every new bird → modify existing code
- Logic becomes scattered and fragile

3. Behavior Controlled by Data (Type Code Smell)

Instead of object decides behavior, we have **String (type)** decides behavior. This is a weak design approach. This is known as a **"Type**

Code / Type Checking" design smell.

STEP 1 — Identify SRP Violation

SRP Rule: A class should have **only one reason to change.**

SRP Violations in Bird

1. Multiple Responsibilities

Here Bird class has multiple reasons to change.

2. Conditional-driven Behavior (if/else)

Bird is deciding:

- **WHICH** behavior to execute
- **HOW** to execute it

This mixes:

- Decision logic
- Business logic

3. Monster Method (Growing / Fragile Method)

A method does more than what its name suggests.

Fix: Break into smaller, focused methods or delegate responsibility.

In this example:

- fly() keeps growing as new birds are added
- Increasing conditional complexity
- Hard to maintain

Note:
SRP is **subjective** — depends on context and level of abstraction.

SRP FIX — Version 1

Bird becomes abstract because it defines **common structure and behavior**, while leaving **varying behavior (like fly()) to concrete subclasses**. We remove type-based behavior because **type** was being used to **CONTROL** behavior. That's wrong design. Keep only **common abstraction.**

```
public abstract class Bird {
    double weight;
    String color;
    double size;

    abstract void fly();

    void eat() {
        System.out.println("Bird is eating");
```

```
    }

    void makeSound() {
        System.out.println("Bird is making sound");
    }
}
```

Use Polymorphism : Replace type with polymorphism.

Concrete Classes

```
public class Crow extends Bird {
    @Override
    void fly() {
        System.out.println("Bird is flying.");
    }
}
```

```
public class Pigeon extends Bird {
    @Override
    void fly() {
        System.out.println("Bird is flying.");
    }
}
```

```
public class Sparrow extends Bird {
    @Override
    void fly() {
        System.out.println("Bird is flying.");
    }
}
```

Each class now defines its own behavior, instead of relying on conditional logic inside a single class.

Edge Cases

```
public class Ostrich extends Bird {
    @Override
    void fly() {
        throw new UnsupportedOperationException("Bird cannot
fly.");
    }
}
```

```
public class Penguin extends Bird {
    @Override
    void fly() {
        throw new UnsupportedOperationException("Bird cannot
fly.");
    }
}
```

Common Incorrect Fixes

Option 1 — Empty method

```
void fly() {}
```

- Silent failure
- Bug-prone

Option 2 — Exception

```
void fly() {
    throw new UnsupportedOperationException("Cannot fly");
}
```

- Breaks expected behavior
- Client must handle exceptions

Option 3 — Log message

```
System.out.println("Cannot fly");
```

- Not real behavior
- Hides design flaw

The issue is not just how **fly()** is implemented, but how we modelled flying in the abstraction itself.

Main Method

```
public class Main {
    public static void main(String[] args) {
        Bird sparrow = new Sparrow();
        sparrow.makeSound();
        sparrow.eat();
        sparrow.fly();
        System.out.println();

        Bird penguin = new Penguin();
        penguin.makeSound();
        penguin.eat();
        penguin.fly(); // UnsupportedOperationException: Bird
cannot fly
```

```
    }
}

/* *
    Output :
        Bird is making sound
        Bird is eating
        Bird is flying.

        Bird is making sound
        Bird is eating
        // UnsupportedOperationException: Bird cannot fly
* */
```

What we achieved

- Removed conditional logic
- Introduced polymorphism

New Problem

- Some birds cannot fly → abstraction is wrong → Forced behavior
- Code duplication (same flying logic)
- Not extensible

-

STEP 3 — OCP (Fix Behavior Design) — Version 2

Software should be:

- **Open for extension** → we can add new features
- **Closed for modification** → we don't change existing code

Extract Flying Strategy

Flying is a **behavior**, not a **type**.

```
public interface FlyingStrategy {
    void fly();
}
```

```
public class CanFly implements FlyingStrategy {
    @Override
    public void fly() {
        System.out.println("Bird is flying");
    }
}
```

```
public class CannotFly implements FlyingStrategy {
    @Override
    public void fly() {
        System.out.println("Bird cannot fly");
    }
}
```

-

STEP 4 — COMPOSITION

Instead of inheritance, we **compose behavior**

```
public abstract class Bird {
    double weight;
    String color;
    double size;

    protected FlyingStrategy flyingStrategy;

    public Bird(FlyingStrategy flyingStrategy) {
        this.flyingStrategy = flyingStrategy;
    }

    void performFly() {
        flyingStrategy.fly();
    }

    void eat() {
        System.out.println("Bird is eating");
    }

    void makeSound() {
        System.out.println("Bird is making sound");
    }
}
```

Update Concrete Classes

```
public class Crow extends Bird {
    public Crow() {
        super(new CanFly());
    }
}


public class Pigeon extends Bird {
    public Pigeon() {
        super(new CanFly());
    }
}


public class Sparrow extends Bird {
    public Sparrow() {
        super(new CanFly());
    }
}


public class Ostrich extends Bird {
    public Ostrich() {
        super(new CannotFly());
    }
}


public class Penguin extends Bird {
    public Penguin() {
        super(new CannotFly());
    }
}
```

Main method

```
public class Main {
    public static void main(String[] args) {
        //To test all birds
        Bird[] birds = {
                new Sparrow(),
                new Crow(),
                new Pigeon(),
                new Penguin(),
                new Ostrich()
        };

        for (Bird bird : birds) {
            bird.performFly();
        }
    }
}

/* *
    Output :
        Bird is flying
        Bird is flying
        Bird is flying
        Bird cannot fly
        Bird cannot fly
* */
```

Outcome

- No modification needed
- Behavior reusable

- Extensible design

-

STEP 5 — Add Eating Behavior — Version 3

Until now, we assumed eating is common.
But consider:

- Some birds eat veg food
- Some birds eat non-veg food

Eating is also a varying behavior.

```
public interface EatingStrategy {
    void eat();
}
```

```
public class VegEating implements EatingStrategy {
    @Override
    public void eat() {
        System.out.println("Eating vegetables");
    }
}
```

```
public class NonVegEating implements EatingStrategy {
    @Override
    public void eat() {
```

```
        System.out.println("Eating non-vegetarian food.");
    }
}
```

Update Bird

```
public abstract class Bird {
    double weight;
    String color;
    double size;

    protected FlyingStrategy flyingStrategy;
    protected EatingStrategy eatingStrategy;

    public Bird(FlyingStrategy flyingStrategy, EatingStrategy
eatingStrategy) {
        this.flyingStrategy = flyingStrategy;
        this.eatingStrategy = eatingStrategy;
    }

    void performFly() {
        flyingStrategy.fly();
    }

    void performEat() {
        eatingStrategy.eat();
    }

    void makeSound() {
        System.out.println("Bird is making sound");
    }
}
```

Update Concrete Classes

```
public class Crow extends Bird {
    public Crow() {
        super(new CanFly(), new VegEating());
    }
}


public class Pigeon extends Bird {
    public Pigeon() {
        super(new CanFly(), new NonVegEating());
    }
}


public class Sparrow extends Bird {
    public Sparrow() {
        super(new CanFly(), new VegEating());
    }
}


public class Ostrich extends Bird {
    public Ostrich() {
        super(new CannotFly(), new NonVegEating());
    }
}


public class Penguin extends Bird {
    public Penguin() {
        super(new CannotFly(), new NonVegEating());
    }
}
```

Main method

```
public class Main {
    public static void main(String[] args) {
        //To test all birds
        Bird[] birds = {
                new Sparrow(),
                new Crow(),
                new Pigeon(),
                new Penguin(),
                new Ostrich()
        };

        for (Bird bird : birds) {
            System.out.println("Bird " +
bird.getClass().getSimpleName() + ":");
            bird.performFly();
            bird.performEat();
            System.out.println();
        }
    }
}

/* *
    Output :
        Bird Sparrow:
        Bird is flying.
        Eating vegetables

        Bird Crow:
        Bird is flying.
        Eating vegetables

        Bird Pigeon:
```

```
        Bird is flying.
        Eating non-vegetarian food.

        Bird Penguin:
        Bird cannot fly.
        Eating non-vegetarian food.

        Bird Ostrich:
        Bird cannot fly.
        Eating non-vegetarian food.
* */
```

OCP Achieved

Add new behavior:

- Behavior is extensible
- Reusable strategies
- No modification needed
- No existing code change
- Small, focused interfaces
- No unnecessary implementation

Step 6 - Liskov Substitution Principle (LSP)

Any child class should be usable wherever the parent is used, without special checks or breaking behavior.

```
Bird b = new Penguin(); // should work correctly
```

Client should NOT need:

- instanceof
- special conditions
- try/catch
- runtime surprises

Problem in Earlier Design

After SRP:

```
abstract class Bird {
    abstract void fly();
}
```

Penguin Implementation

```
class Penguin extends Bird {
    void fly() {
        throw new UnsupportedOperationException("Cannot fly");
    }
}
```

Problem

```
Bird b = new Penguin();
b.fly();  // Runtime crash
```

- Parent (Bird) implies: “All birds can fly”, Child (Penguin) says: “I cannot fly”

- Client expects Bird.fly() to work
- Penguin breaks that expectation

Violates LSP - Child is **breaking the contract of parent**

We made wrong assumption:
Bird → always fly
That assumption is wrong

Most people try to fix Penguin. But real fix is: **fix abstraction.**

After Strategy

```
Bird b = new Penguin();
b.performFly();  // Safe
```

Why LSP is now satisfied

- No exception
- No fake implementation
- No special handling
- Behavior is correct

-

Step 5 - Interface Segregation Principle (ISP)

Don’t force a class to implement methods it doesn’t need. Better to have **multiple small interfaces** than **one large (fat) interface.**

Problem (Fat Interface)

```
interface Bird {
    void fly();
    void eat();
    void speak();
}
```

Issues

- Classes forced to implement unused methods
- Dummy implementations

Fix — Split Interfaces

```
interface FlyingStrategy {
    void fly();
}
```

```
interface EatingStrategy {
    void eat();
}
```

- Interfaces should be small and focused

- Each represents one behavior

Why ISP is satisfied now

- No class forced to implement unused methods
- No dummy methods
- Clean, focused contracts
- Behavior is optional & pluggable

ISP says that no class should be forced to implement methods it doesn’t use. In our design, instead of creating a fat Bird interface with fly(), eat(), etc., we split behaviors into small interfaces like FlyingStrategy and EatingStrategy. This ensures flexibility and avoids dummy implementations.

Bird (identity)
  |
  |--- uses ---> FlyingStrategy
  |--- uses ---> EatingStrategy

-

Step 6 - Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules. Both

should depend on abstractions.

Don’t do this:

Bird → NormalFlying (direct dependency)

```
class Bird {
    CanFly canFly = new CanFly();
}
```

Problems

- Tight coupling to NormalFlying
- Not flexible (changing behavior → modify Bird)
- Hard to test (no mocking possible)
- Hard to Extend, New behavior → changes in existing code

Do this:

Bird → FlyingStrategy (abstraction)

```
abstract class Bird {
    protected FlyingStrategy flyingStrategy;
}
```

What We Achieved

- Loose coupling
- Flexible design
- Better testability

- Easier to extend

-

Step 7 - Dependency Injection (DI)

Instead of creating dependencies inside a class, we pass them from outside.

Without DI: Bird → creates NormalFlying

Real-Life Analogy: Chef grows vegetables, cooks food, serves food - doing everything.

```
public class Sparrow extends Bird {
    public Sparrow() {
        super(new CanFly(), new VegEating());
    }
}
```

Issues

- Tight coupling
- Hard to change behavior
- Difficult to test
- Violates DIP

With DI: Bird → receives FlyingStrategy

Real-Life Analogy: Someone gives ingredients to chef. Chef just cooks.

Constructor Injection

```
class Sparrow extends Bird {
    public Sparrow(FlyingStrategy flyingStrategy) {
        super(flyingStrategy);
    }
}
```

Main method

```
public class Main {
    static void main() {
        //To test all birds
        Bird[] birds = {
                new Sparrow(new CanFly(), new VegEating()),
                new Crow(new CanFly(), new VegEating()),
                new Pigeon(new CanFly(), new NonVegEating()),
                new Penguin(new CannotFly(), new NonVegEating()),
                new Ostrich(new CannotFly(), new NonVegEating())
        };

        for (Bird bird : birds) {
            System.out.println("Bird " +
bird.getClass().getSimpleName() + ":");
            bird.performFly();
```

```
            bird.performEat();
            System.out.println();
        }
    }
}

/* *
    Output :
        Bird Sparrow:
        Bird is flying.
        Eating vegetables

        Bird Crow:
        Bird is flying.
        Eating vegetables

        Bird Pigeon:
        Bird is flying.
        Eating non-vegetarian food.

        Bird Penguin:
        Bird cannot fly.
        Eating non-vegetarian food.

        Bird Ostrich:
        Bird cannot fly.
        Eating non-vegetarian food.
* */
```

Benefits

- Easy to change behavior
- Easy to test
- No tight coupling

Another option : Setter Injection

```
class Bird {
    protected FlyingStrategy flyingStrategy;

    public void setFlyingStrategy(FlyingStrategy flyingStrategy) {
        this.flyingStrategy = flyingStrategy;
    }
}
```

Usage

```
Bird b = new Sparrow();
b.setFlyingStrategy(new NormalFlying());
```

Problem

- Object can be in invalid state (strategy not set)

DIP vs DI

DIP → WHAT to depend on (abstraction)

DI → HOW to provide it

Final Mental Model

Main (creates objects)
↓
injects
↓
Bird (uses abstraction)
↓
FlyingStrategy, EatingStrategy

- Behavior selection → done via DI
- Pattern used → **Strategy Pattern + DI**

Summary

1. **Start with SRP** → separate responsibilities
2. Move to **OCP** → avoid modification using Strategy
3. Introduce **composition over inheritance**
4. Fix **LSP** → no forced behavior
5. Apply **ISP** → small interfaces
6. Apply **DIP + DI** → inject behaviors